실습일지 — 지도 기반 노선 타당성 분석 시뮬레이터

컴퓨터시스템공학과 3학년 2학기 · 202245079 신선호
심화응용프로젝트 10주차 · 2026년 11월 8일 (지난 일지 11월 2일 이후)

1. 지금 화면



서울~부산 고가 노선(11월 7일 실제 화면). 중간 역마다 행정동 이름이 붙고(전에는 206개가 이름 없는 번호), 패널은 35‰보다 가파른 산 구간 27곳과 「뚫기·돌아가기」 두 안의 비용·시간을 보여 줍니다.

2. 이번에 한 일

54개	262 → 290	116 → 219	6 → 42
커밋	ETL 테스트	백엔드 테스트	프론트엔드 테스트

과제	결말
노선 수요	찍은 점 둘레만 세던 것을 **선 전체**로 — 수요 식을 노선 단위로 다시 학습
산 구간	35‰로 판정해 **뚫는 안·돌아가는 안**을 나란히 제안
제안서 빈칸	소셜 로그인 화면, 관리자 회원·통계·단가, 차트, Swagger, Redis, Docker 배포 — **EC2 업로드만 남음**
보안	권한을 DB에서 읽기, 로그인 횟수 제한, 노선 1,000km 상한, 열려 있던 Redis 포트 닫기
복선전철 수요 하한	여전히 막힘 — 승하차에 상·하행 구분이 없음(KRIC 키 필요)

3. 주요 작업

① 노선 수요를 선 전체로

- 수요를 노선에 **찍은 점 둘레 1km만** 세고 있었습니다 — 경유점을 많이 찍을수록 수요가 늘었습니다
- 강남~잠실 추정 **7.9만** 명 대 2호선 실측 **66.4만** 명
- 운영 노선 26개로 다시 학습: R^2 **0.552 → 0.834**, 오차 **1.75 → 1.44배**
- 인구 자료를 전국 252개 시군구로 넓혀 이름 없는 역 **206개 → 0개**

② 산을 만나면 두 안

- 지형 타일 1,312장으로 땅높이를 읽어 **35‰**를 지키는 선로를 구하고, 터널과 우회로를 함께 찾습니다
- 지형 자료가 건물 꼭대기까지 재서 역삼역 빌딩 숲이 산으로 잡혔습니다 → 0.5km 중앙값으로 해결
- 돌면 오히려 비싼 곳은 뚫습니다(서울~부산 6곳 중 1곳만 우회). 계산을 Redis에 담아 **1.5초 → 0.08초**

③ 공개 전에 막을 것

- 강등된 관리자의 옛 토큰이 **720분** 동안 관리자로 통함 → 권한을 DB에서 읽도록
- 18,528km 노선 하나에 계산 **137.8초** → 1,000km 상한
- 로그인 15분에 10번 제한 — 주소 헤더를 바꿔 비켜 가는 것도 막히는지 확인

4. 「노선 수요」의 정의와 정한 이유

항목	기준
노선 둘레	그린 선을 0.5km마다 잰 점의 반경 1km 안 인구·종사자를 **겹침 없이** 합친 값
수요 식	log(일이용객) = -2.499 + 0.471·log(인구) + 0.691·log(종사자) (수도권 운영 노선 26개)
그러면	같은 수요를 모든 수단에 주고, 수단은 건설비·속도로 가릅니다

노선	전 (점 둘레)	지금 (선 전체)	추천 수단
서울시청~잠실	6.5만 명/일	28.7만 명/일	그대로
강남~잠실	7.9만 명/일	29.9만 명/일	그대로
서울~부산	5.4만 명/일	53.9만 명/일	그대로

이유

- 역 하나로 배운 식을 노선에 쓰면 **단위가 맞지 않습니다**
- 역마다 돌려 더하는 안을 실제로 재 보니 BRT가 **B/C 23** — 역을 쪼갤수록 합이 커지는 식이라 버렸습니다
- 노선 단위는 역 수에 흔들리지 않고 오차도 더 작았습니다

한계

- 표본이 **수도권 26개**뿐이라 지방·장거리는 외삽입니다
- 수요가 수단과 무관해 값싼 BRT의 B/C가 19.6처럼 크게 나옵니다 — **절대값이 아니라 순위**를 읽어야 합니다

5. 느낀 점

- **바꾸기 전에 재야 합니다.** 그럴듯했던 역별 합산을 재지 않고 넣었다면 발표 숫자가 전부 틀렸을 것입니다
- **문서도 믿으면 안 됩니다.** 노트의 「노선 반경 1km」, 테스트의 「포트를 감시한다」 모두 코드와 달랐습니다
- **내 PC 통과가 끝이 아닙니다.** 한국 시각으로 적은 값이 UTC인 CI 서버에서는 9시간 뒤의 미래가 되어 테스트가 깨졌습니다